

lab02_leila_kawanishi_277185

Leila Kawanishi

#Laboratório 2: Processamento de Bases de Dados em Lote Benilton Carvalho & Guilherme Ludwig

##Introdução A coleta de dados em épocas mais recentes teve seu custo reduzido e volume ampliado significativamente. Redes sociais combinadas com dispositivos móveis rastreiam bilhões de pessoas no mundo, com informações das mais diversas origens: rotinas de passeio, amizades, históricos de compras, históricos de localização, fotografias geo-referenciadas, etc (FYI: com o escândalo da Cambridge Analytica, surgiram evidências de que o Facebook tenha registros de histórico de chamadas e SMS, para mensagem e ligações ocorridas fora do aplicativo Facebook, em telefones Android: <https://www.theverge.com/2018/3/25/17160944/facebook-call-history-sms-data-collection-android>). Empresas, por meio de programas de fidelidade, aprendem hábitos de compras, propensão de compras de certos produtos dependendo de horários, produtos que são comprados juntos e informações financeiras referentes a pagamentos. Desta maneira, o volume das bases de dados cresce vertiginosamente, de modo que é necessária a mudança de paradigma na estatística moderna: os dados não mais podem trafegar até o analista de dados; a análise deve ser transportada até os dados.

##Objetivos Ao fim deste laboratório, você deve ser capaz de:

- Importar um arquivo volumoso por partes;
- Calcular estatísticas suficientes para métrica de interesse em cada uma das partes importadas;
- Manter em memória apenas o conjunto de estatísticas suficientes e utilizar a memória remanescente para execução de cálculos;
- Combinar as estatísticas suficientes de modo a compor a métrica de interesse.
- Tarefa (retirada dos últimos slides da última aula)
- Lendo 100.000 observações por vez (se você acredita que seu computador não tem memória suficiente, utilize 10.000 observações por vez), determine o percentual de vôos por Cia. Aérea que apresentou atraso na chegada (ARRIVAL_DELAY) superior a 10 minutos. As companhias a serem utilizadas são: AA, DL, UA e US. A estatística de interesse deve ser calculada para cada um dos dias de 2015. Para a determinação deste percentual de atrasos, apenas verbos do pacote dplyr e comandos de importação do

pacote readr podem ser utilizados. Os resultados para cada Cia. Aérea devem ser apresentados em um formato de calendário.

Observação: a atividade descrita no slide pede para ler apenas 100 registros por vez. Aqui, mudamos para 100 mil registros por vez, para que haja melhor aproveitamento do tempo em classe.

#Instruções

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada ($ARRIVAL_DELAY > 10$)?

Resposta: O percentual de voos ($ARRIVAL_DELAY > 10$) é dado pela $[(\text{numero de voos atrasados})/(\text{numero total de voos})] * 100$, neste caso as estatísticas suficientes para o cálculo são (numero de voos atrasados) e (numero total de voos). A ideia é diminuir as informações que o computador precisa lidar, ao filtrar os casos de atraso já gera economia no processamento.

2. Crie uma função chamada getStats que, para um conjunto de qualquer tamanho de dados provenientes de flights.csv.zip, execute as seguintes tarefas (usando apenas verbos do dplyr:
 - a. Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;
 - b. Remova observações que tenham valores faltantes em campos de interesse;
 - c. Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
 - d. Para cada grupo em b., determine as estatísticas apontadas no item 1. e os retorne como um objeto da classe tibble;
 - e. A função deve receber apenas dois argumentos:
 - input: o conjunto de dados (referente ao lote em questão);
 - pos: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
library(readr)

dados <- read_csv("C:/Users/USER/Documents/ME315/flights.csv.zip", n_max = 10)
```

Rows: 10 Columns: 31

-- Column specification -----

Delimiter: ","

chr (10): AIRLINE, TAIL_NUMBER, ORIGIN_AIRPORT, DESTINATION_AIRPORT, SCHEDUL...

dbl (15): YEAR, MONTH, DAY, DAY_OF_WEEK, FLIGHT_NUMBER, DEPARTURE_DELAY, TAX...

```
lgl (6): CANCELLATION_REASON, AIR_SYSTEM_DELAY, SECURITY_DELAY, AIRLINE_DEL...
```

i Use ``spec()`` to retrieve the full column specification for this data.

i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
head(dados)
```

```
# A tibble: 6 x 31
```

	YEAR	MONTH	DAY	DAY_OF_WEEK	AIRLINE	FLIGHT_NUMBER	TAIL_NUMBER	ORIGIN_AIRPORT
	<dbl>	<dbl>	<dbl>	<dbl>	<chr>	<dbl>	<chr>	<chr>
1	2015	1	1	4	AS	98	N407AS	ANC
2	2015	1	1	4	AA	2336	N3KUAA	LAX
3	2015	1	1	4	US	840	N171US	SFO
4	2015	1	1	4	AA	258	N3HYAA	LAX
5	2015	1	1	4	AS	135	N527AS	SEA
6	2015	1	1	4	DL	806	N3730B	SFO

```
# i 23 more variables: DESTINATION_AIRPORT <chr>, SCHEDULED_DEPARTURE <chr>,  
# DEPARTURE_TIME <chr>, DEPARTURE_DELAY <dbl>, TAXI_OUT <dbl>,  
# WHEELS_OFF <chr>, SCHEDULED_TIME <dbl>, ELAPSED_TIME <dbl>, AIR_TIME <dbl>,  
# DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <dbl>, SCHEDULED_ARRIVAL <chr>,  
# ARRIVAL_TIME <chr>, ARRIVAL_DELAY <dbl>, DIVERTED <dbl>, CANCELLED <dbl>,  
# CANCELLATION_REASON <lgl>, AIR_SYSTEM_DELAY <lgl>, SECURITY_DELAY <lgl>,  
# AIRLINE_DELAY <lgl>, LATE_AIRCRAFT_DELAY <lgl>, WEATHER_DELAY <lgl>
```

```
library(dplyr)
```

Anexando pacote: 'dplyr'

Os seguintes objetos são mascarados por 'package:stats':

```
filter, lag
```

Os seguintes objetos são mascarados por 'package:base':

```
intersect, setdiff, setequal, union
```

```

getStats <- function(input, pos){

  input %>%
    filter(AIRLINE %in% c("AA", "DL", "UA", "US"), # filtra Cias. Aéreas: AA, DL, UA e US
           !is.na(DAY), #filtra NA nos dias
           !is.na(MONTH), #filtra NA no mes
           !is.na(ARRIVAL_DELAY)) %>% #filtra NA no atraso

  group_by(DAY, MONTH, AIRLINE) %>% #Agrupar por dia, mes e cia aerea
  summarise(
    n = n(), # conta quantas observações existem em cada grupo
    atrasados = sum(ARRIVAL_DELAY > 10) #variael que guarda o numero de TRUE para a condiçã
  )

}

```

3. Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.

- Configure o tamanho do lote (chunk) para 100 mil registros;
- Configure a função de callback para instanciar DataFrames aplicando a função `getStats` criada acima;
- Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```
spec(dados)
```

```

cols(
  YEAR = col_double(),
  MONTH = col_double(),
  DAY = col_double(),
  DAY_OF_WEEK = col_double(),
  AIRLINE = col_character(),
  FLIGHT_NUMBER = col_double(),
  TAIL_NUMBER = col_character(),
  ORIGIN_AIRPORT = col_character(),
  DESTINATION_AIRPORT = col_character(),
  SCHEDULED_DEPARTURE = col_character(),
  DEPARTURE_TIME = col_character(),
  DEPARTURE_DELAY = col_double(),
  TAXI_OUT = col_double(),

```

```

WHEELS_OFF = col_character(),
SCHEDULED_TIME = col_double(),
ELAPSED_TIME = col_double(),
AIR_TIME = col_double(),
DISTANCE = col_double(),
WHEELS_ON = col_character(),
TAXI_IN = col_double(),
SCHEDULED_ARRIVAL = col_character(),
ARRIVAL_TIME = col_character(),
ARRIVAL_DELAY = col_double(),
DIVERTED = col_double(),
CANCELLED = col_double(),
CANCELLATION_REASON = col_logical(),
AIR_SYSTEM_DELAY = col_logical(),
SECURITY_DELAY = col_logical(),
AIRLINE_DELAY = col_logical(),
LATE_AIRCRAFT_DELAY = col_logical(),
WEATHER_DELAY = col_logical()
)

```

```

banco1 <- read_csv_chunked(
  "flights.csv.zip",
  callback = DataFrameCallback$new(getStats), #separa os lotes e aplica o getStats em cada
  chunk_size = 100000, #tamanho do chunk
  col_types = cols_only(
    AIRLINE = col_character(),
    DAY = col_integer(),
    MONTH = col_integer(),
    ARRIVAL_DELAY = col_double()
  )
)

```

```

`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.

```

[illegible]

```
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
`summarise()` has regrouped the output.
i Summaries were computed grouped by DAY, MONTH, and AIRLINE.
i Output is grouped by DAY and MONTH.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(DAY, MONTH, AIRLINE))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

```
head(banco1)
```

```
# A tibble: 6 x 5
# Groups:   DAY, MONTH [2]
  DAY MONTH AIRLINE      n atrasados
  <int> <int> <chr>    <int>    <int>
1     1     1 AA      1379      514
2     1     1 DL      1558      132
3     1     1 UA      1269      367
4     1     1 US      1028      201
5     2     1 AA      1508      649
6     2     1 DL      2261      406
```

```
dim(banco1)
```

```
[1] 1478    5
```

```
glimpse(banco1)
```

```
Rows: 1,478
Columns: 5
Groups: DAY, MONTH [365]
$ DAY      <int> 1, 1, 1, 1, 2, 2, 2, 2, 3, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5, 5, ~
$ MONTH    <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
$ AIRLINE  <chr> "AA", "DL", "UA", "US", "AA", "DL", "UA", "US", "AA", "DL", ~
$ n        <int> 1379, 1558, 1269, 1028, 1508, 2261, 1411, 1174, 1425, 2021, ~
$ atrasados <int> 514, 132, 367, 201, 649, 406, 493, 229, 767, 702, 699, 377, ~
```

4. Crie uma função chamada computeStats que:

- Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
- Retorne as informações em um tibble contendo apenas as seguintes colunas: Cia: sigla da companhia aérea; Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`); Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo $[0,1]$.

```
computeStats <- function(stats){

  stats %>%
    group_by(DAY, MONTH, AIRLINE) %>%
    summarise(
      n = sum(n),
      atrasados = sum(atrasados),
      .groups = "drop" #nao mantem as colunas de agrupamento depois do calculo
    ) %>%
    mutate(prop = atrasados / n,
           Data = as.Date(paste(2015, MONTH, DAY, sep = "-"))) %>%
    rename(Cia = AIRLINE,
           Perc = prop) %>%
    select(Cia, Data, Perc)
}

resultado <- computeStats(banco1)

head(resultado)
```

```
# A tibble: 6 x 3
  Cia    Data      Perc
  <chr> <date>    <dbl>
1 AA    2015-01-01 0.373
2 DL    2015-01-01 0.0847
3 UA    2015-01-01 0.289
4 US    2015-01-01 0.196
5 AA    2015-02-01 0.201
6 DL    2015-02-01 0.203
```

5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

- Instale e carregue os pacotes `ggcal` e `ggplot2`.
- Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.

- c. Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá:
- Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`.
 - Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`.
 - Retornar para o usuário a base do calendário criada acima.
- d. Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.
- e. Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Título")`.

```
# install.packages("remotes")
# remotes::install_github("jayjacobs/ggcal")
# install.packages("ggplot2")
library(ggcal)
library(ggplot2)

pal <- scale_fill_gradient(low = "#4575b4", high = "#d73027")

baseCalendario <- function(stats, cia){

  dados_cia <- stats %>%
    filter(Cia == cia) %>%
    select(Data, Perc)

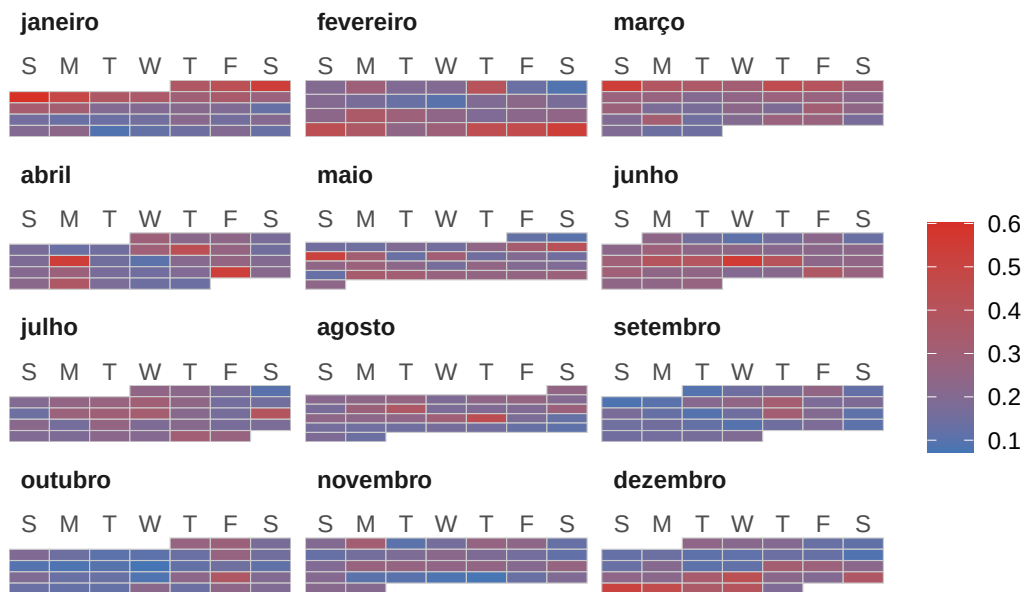
  ggcal(dates = dados_cia$Data, fills = dados_cia$Perc)

}

cAA <- baseCalendario(resultado, "AA")
cDL <- baseCalendario(resultado, "DL")
cUA <- baseCalendario(resultado, "UA")
cUS <- baseCalendario(resultado, "US")
```

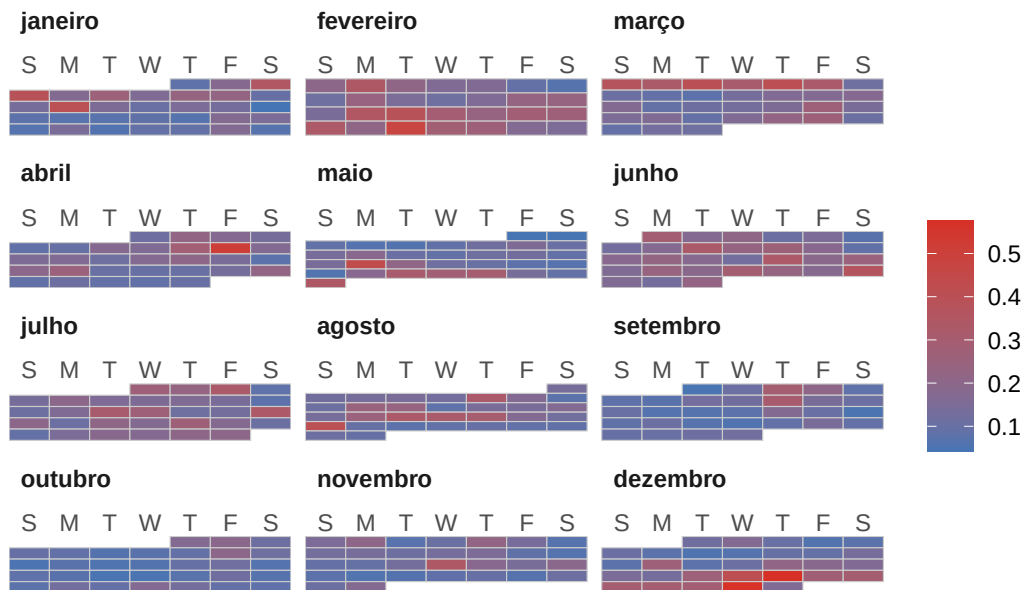
```
cAA + pal + ggtitle("AA")
```

AA



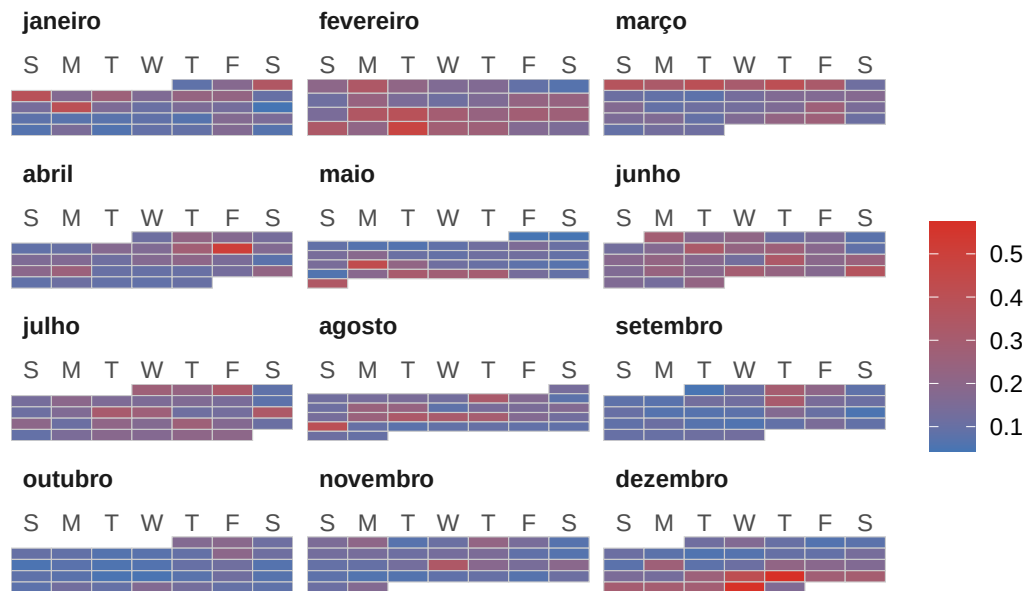
```
cDL + pal + ggtitle("DL")
```

DL



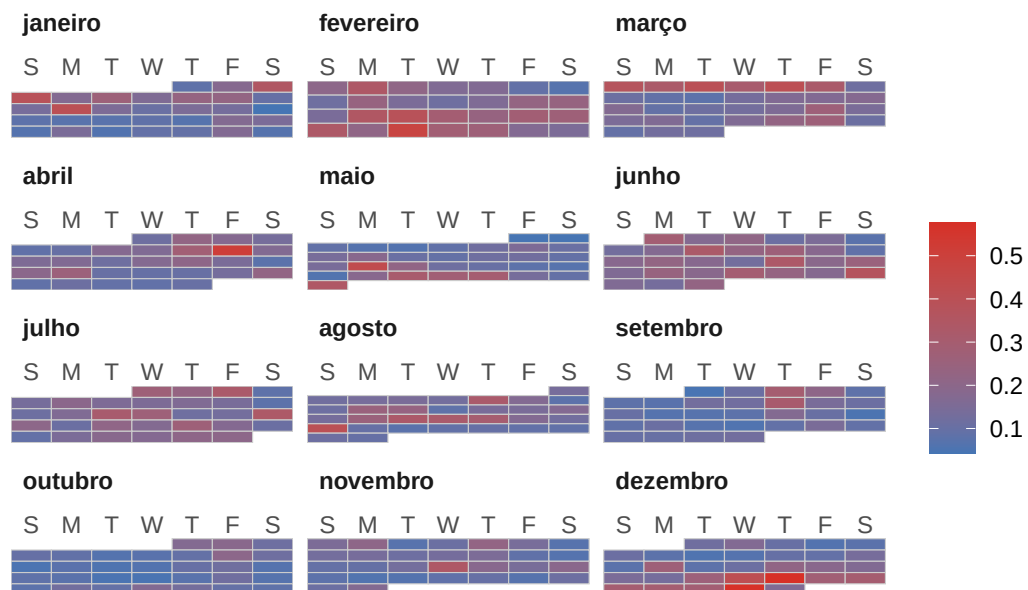
```
cDL + pal + ggtitle("UA")
```

UA



```
cDL + pal + ggtitle("US")
```

US



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-23 17:05:10 -03"
```